

Lab 1: Evaluation of Continuous Batching

Shaoyu Wang
wangshao@usc.edu

1 SETTINGS

This section describes the hardware testbed, model and serving system configuration, and the workload used in our evaluation. Together, these elements define the environment in which we study the effectiveness of continuous batching.

Testbed. The experiments were conducted on a server with an Intel Xeon Platinum 8275 CPU, featuring 96 cores and a base clock speed of 3.00 GHz. The system is running Ubuntu 22.04 LTS and is equipped with 1.1 TiB of DRAM. For GPU acceleration, the server houses eight **NVIDIA A100-SXM4-40GB GPUs** connected via NVLink, each with 40 GiB of memory. In this lab, only **one A100-40GB GPU** will be used for small scale experiments.

Model and Baseline. The chosen Large Language Model was **Llama-3-8B**. It is composed of 32 transformer layers. Each layer has a hidden size of 4096 and employs 32 attention heads. Due to its use of Grouped-Query Attention (GQA), the model also has 8 key/value heads, which are shared across the attention heads. This model was selected due to its compact size, making it a suitable candidate for running on a single A100 GPU without memory constraints. The serving system for this experiment is **SGLang**, a framework with clear code organization. This choice was made to leverage my existing knowledge with the SGLang codebase, which simplifies the process of locating and modifying the batching logic for the subsequent steps of the evaluation.

Workload. The input workload was constructed using prompts from the ShareGPT dataset, which reflects realistic user interactions with conversational models. In total, **200 prompts** were sampled, each allows up to a **context length of 1024 tokens** to ensure consistency across requests. These requests were injected into the serving system at a steady arrival rate of **five requests per second**, which creates a moderate but sustained pressure on the batching mechanism. This configuration was selected to provide a tractable workload that can be executed within the limited resources

of a single GPU, while still exposing meaningful batching behaviors under concurrent request patterns.

This experimental setup is both realistic and feasible, enabling a clear evaluation of continuous batching under controlled conditions.

2 IMPLEMENTATION

By default, SGLang employs a continuous batching scheduler. To evaluate the effectiveness of this mechanism, we introduced several modifications that enable non-continuous batching and facilitate the collection of detailed performance metrics. These changes can be summarized as follows:

- **Non-continuous batching option.** We added a configuration option to disable continuous batching. When this option is enabled, the scheduler no longer admits new requests into an ongoing batch. Instead, any incoming requests are deferred until the current batch completes. Once the current batch has been fully decoded, the scheduler admits a new batch composed of pending requests from the queue, subject to GPU memory availability for KV cache allocation and the configured maximum prefill length of the system. The admitted requests are then prefilled and decoded in subsequent steps.
- **Iteration tracking.** At the beginning of each benchmark run, an iteration counter is initialized to zero and incremented at every execution step. Meanwhile, a list is maintained to record the batch size at each step, allowing us to visualize the dynamics of batch formation and execution over time.
- **Request-level monitoring.** A request tracker was implemented to record fine-grained request-level information. For each request, we log its arrival time, scheduling time, and completion time, together with the iteration indices corresponding to these events. Additionally, inter-token latency (ITL) is measured by recording the time between consecutive token outputs for every request.

Metric	Non-continuous	Continuous
Throughput (tok/s)	321.7	732.5
Mean TTFT (s)	32.2	0.1
Median TTFT (s)	32.7	0.0
P99 TTFT (s)	66.2	0.2
Mean ITL (ms)	14	17
Median ITL (ms)	14	15
P99 ITL (ms)	16	69

Table 1: Performance comparison of continuous vs. non-continuous batching in SGLang.

All data collection is performed throughout the duration of the benchmark. The recorded traces are written to disk upon completion of each run for subsequent analysis and visualization.

3 EVALUATION

We evaluate continuous batching against conventional (non-continuous) batching along three dimensions. First, we quantify the direct performance improvements achieved by continuous batching. Second, we examine the run-time dynamics to understand system-level behavior. Finally, we conduct a request-level analysis to highlight its efficiency in handling individual requests.

In our experiments, the maximum number of prefill tokens per execution step is set to 4096. CUDA graphs are enabled to minimize kernel launch overhead and to push system performance closer to hardware limits.

3.1 Serving Performance

Table 1 presents a comparison between continuous and non-continuous batching in terms of throughput and latency metrics. Several clear trends emerge from the results.

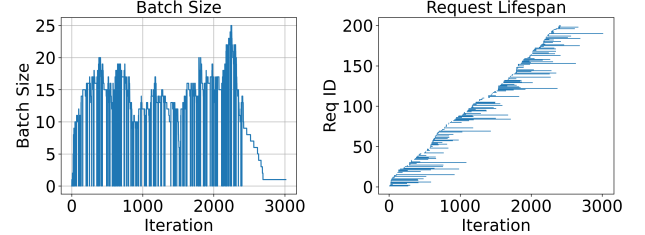
Continuous batching delivers almost twice the throughput. This improvement can be attributed to the ability of continuous batching to dynamically admit new requests into ongoing decoding, thereby maintaining higher GPU utilization.

The time to first token (TTFT) is drastically reduced under continuous batching. The mean TTFT decreases from 32.2 seconds in the non-continuous case to just 0.1 seconds. Similarly, the median and 99th percentile TTFT values are reduced by two to three orders of magnitude. These results highlight the responsiveness advantage

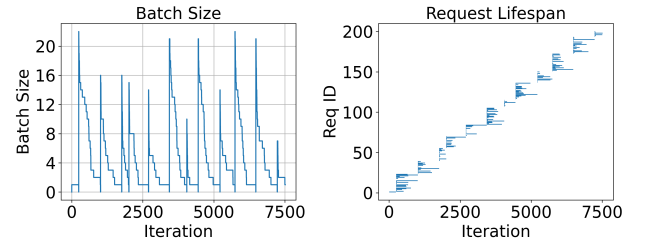
of continuous batching: by allowing requests to join in-flight decoding steps, new requests experience minimal queuing delays before producing their first token.

Inter-token latency (ITL) exhibits a more nuanced behavior. The mean and median ITL remain within a similar range across both configurations, indicating that the steady-state token generation rate is not strongly influenced by the batching strategy. Non-continuous batching shows slightly lower ITL, which can be attributed to its simpler scheduling mechanism and reduced overhead. However, the P99 ITL is markedly higher under continuous batching (69 ms versus 16 ms). This long-tail effect likely arises from interleaved prefilling during decoding, which can delay the emission of subsequent tokens for requests in the ongoing batch.

Overall, the results demonstrate that continuous batching greatly improves throughput and reduces TTFT, which is critical for interactive applications. With heavier request traffic and longer context length, the effect of continuous batching can be even more significant. The trade-off is a tail problem in inter-token latency, which should inspire further investigation into scheduling policies.



(a) Continuous batching



(b) Non-continuous batching

Figure 1: Left shows the batch size of each iteration (a drop to 0 indicates a prefill batch). Right shows the lifetime of each request over iterations.

3.2 Batching Characteristics

Figure 1a compares batching dynamics under continuous and non-continuous policies. The left subplots show batch sizes per iteration (with drops to zero indicating prefills), and the right subplots show request lifespans.

With continuous batching, batch sizes fluctuate as new requests are admitted whenever resources free up, sustaining GPU utilization and producing staggered lifespans where requests overlap in execution. This interleaving reduces queuing delays and improves throughput.

Non-continuous batching instead shows a sawtooth pattern: batches start large, shrink as decoding progresses, then reset after completion. Lifespans form distinct clusters, as requests can only enter at batch boundaries, leaving long idle periods.

To conclude, continuous batching yields higher utilization and smoother prefill-decode overlap, minimizing latency, whereas non-continuous batching causes bursty execution and resource underuse.

3.3 Request Analysis

Figure 2 shows the cumulative distribution of request-level statistics under non-continuous and continuous batching. While request generation times are similar, the two policies yield very different queuing delays and TTFT.

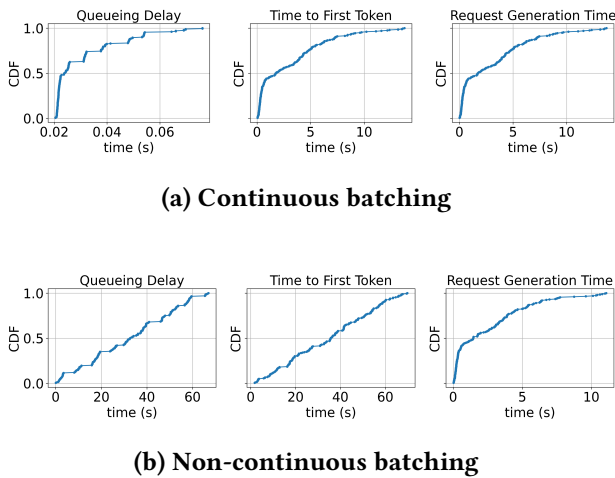


Figure 2: Cumulative distribution function of requests.

For non-continuous batching (Figure 2b), the queuing delay is broadly distributed and exhibits a stair-step pattern, with many requests waiting tens of seconds before execution. This stems from the rigid batching policy: new requests must wait until the current batch completes, causing long idle periods for unlucky arrivals. Consequently, TTFT also extends into the tens of seconds, directly reflecting these delays.

In contrast, continuous batching (Figure 2a) keeps queuing delays consistently low: nearly all requests wait less than a second, most only a few milliseconds. This efficient admission translates to much shorter TTFT, tightly concentrated below 10 seconds. The results highlight the responsiveness of continuous batching, where requests are admitted opportunistically into ongoing decoding steps rather than waiting for batch boundaries.

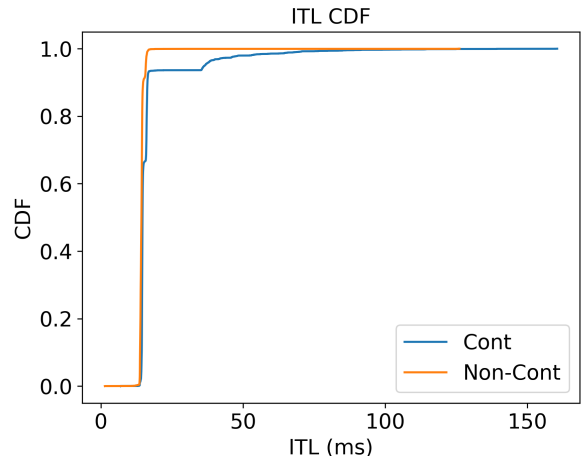


Figure 3: Cumulative distribution function of inter-token latency.

Figure 3 shows the inter-token latency (ITL) CDF under continuous and non-continuous batching. Both achieve sub-10ms ITL for most tokens, but continuous batching introduces a longer tail due to interleaved prefill execution, whereas non-continuous batching yields a tighter distribution by advancing requests in lockstep.